



PROGRAMME DE FORMATION

DevSecOps

Intégrer la sécurité à chaque étape de la chaîne de livraison

57 HEURES

POUR Ingénieurs DevOps et développeurs responsables d'une chaîne de livraison



Mohamed Aziz BEN HAHHA

mohamedazizbenhaha.netlify.app

Présentation

Ce programme ajoute la sécurité à une chaîne DevOps qui fonctionne déjà, étape par étape, en gardant une chaîne rapide. Les participants modélisent les menaces, analysent le code et les dépendances, testent l'application en fonctionnement, puis sécurisent la chaîne d'approvisionnement logicielle : images durcies et analysées, SBOM, signatures, attestations et provenance SLSA vérifiées avant le déploiement. Ils analysent le code d'infrastructure, appliquent des politiques en code, gèrent les secrets avec HashiCorp Vault, et durcissent et surveillent Kubernetes à l'exécution. Le programme s'achève par un projet final : une chaîne avec un contrôle de sécurité à chaque étape et un rapport de sécurité.

Vue d'ensemble

#	PHASE	THÈMES	HEURES
01	Fondamentaux et sécurité applicative	Shift-left, modélisation des menaces, SAST, SCA et licences, DAST	12 h
02	Conteneurs et chaîne d'approvisionnement	Analyse et durcissement des images, SBOM, signature, SLSA, admission	9 h
03	Infrastructure et secrets	Analyse de l'IaC, politiques en code, HashiCorp Vault, détection de secrets	12 h
04	Kubernetes et exécution	Pod Security Standards, durcissement RBAC, NetworkPolicies, Falco	6 h
05	Projet final	Chaîne de livraison sécurisée et rapport de sécurité	9 h
	Réserve	Rattrapage, révision, TP supplémentaires	9 h
Total			57 h

Informations clés

PUBLIC	Ingénieurs DevOps et développeurs responsables d'une chaîne de livraison
PRÉREQUIS	Intégration et livraison continues, Terraform et déploiement sur Kubernetes (cours : Ingénierie DevOps, qui prolonge Docker et Kubernetes), ou équivalent. La construction de pipelines n'est pas réenseignée
DURÉE	57 h : 48 h de chapitres + 9 h de réserve
FORMAT	Cours courts suivis de TP guidés ; une même application et une même chaîne dans toutes les phases
ENVIRONNEMENT DE TP	En local : un portable par participant (16 Go de RAM minimum, Docker, kind, VS Code) ; Jenkins et SonarQube sur un serveur fourni par le formateur ; un dépôt GitHub public et GitHub Container Registry par équipe ; le code Terraform est analysé, pas appliqué ; ni compte cloud ni coût
ÉVALUATION	Un TP validé par phase et un projet final avec rapport de sécurité ; attestation de formation pour les participants qui valident les deux
RÉSULTAT	Les participants savent placer et régler des contrôles de sécurité du code jusqu'à l'exécution sur Kubernetes, et rendre compte du risque résiduel

PHASE 01

12 H

Fondamentaux et sécurité applicative

Les participants apprennent où placer la sécurité dans la chaîne et quel contrôle arrête quoi, modélisent les menaces de l'application du cours avec STRIDE, puis ajoutent l'analyse statique, le contrôle des dépendances et des licences, et l'analyse dynamique de l'application en fonctionnement. TP validé : une chaîne qui échoue sur une injection SQL introduite volontairement, une dépendance vulnérable et une alerte ZAP, avec les faux positifs triés.

#	CHAPITRE	CONTENU	FORMAT	HEURES
1.1	Principes du DevSecOps	Shift-left, modélisation des menaces avec STRIDE, contrôles de sécurité et leurs seuils	Cours + atelier	3 h
1.2	Analyse statique (SAST)	Règles de sécurité SonarQube, règles Semgrep pour les failles d'injection, règles personnalisées, tri des faux positifs	Cours + TP	3 h
1.3	Dépendances et licences (SCA)	OWASP Dependency-Check, mise à jour des dépendances vulnérables, conformité des licences	Cours + TP	3 h
1.4	Tests dynamiques (DAST)	Analyses ZAP baseline, complète et d'API dans la chaîne	Cours + TP	3 h

PHASE 02

9 H

Conteneurs et chaîne d'approvisionnement

Les participants durcissent et analysent les images du cours, puis prouvent leur origine : un SBOM pour chaque image, des signatures et attestations avec Cosign et Sigstore, la provenance SLSA issue du build, et des politiques d'admission qui refusent les images non signées dans le cluster. TP validé : une image non signée refusée par le cluster, tandis que le build signé et attesté est admis.

#	CHAPITRE	CONTENU	FORMAT	HEURES
2.1	Analyse et durcissement des images	Trivy, images de base minimales et distroless, images sans root, correction des vulnérabilités	Cours + TP	3 h
2.2	SBOM et signature	SBOM avec Syft, signature avec Cosign et Sigstore, signature sans clé (keyless)	Cours + TP	3 h
2.3	Attestations et admission	Attestations, provenance SLSA, admission des images signées avec le policy-controller Sigstore	Cours + TP	3 h

PHASE 03

12 H

Infrastructure et secrets

Les participants analysent le code Terraform et Kubernetes du cours avec Checkov et Trivy, imposent des règles dans le cluster avec OPA Gatekeeper ou Kyverno, puis retirent les secrets du code : HashiCorp Vault avec des secrets dynamiques, ses intégrations Jenkins et Kubernetes, et la détection de secrets avec gitleaks. TP validé : une chaîne qui bloque un bucket public dans Terraform et un mot de passe poussé dans le dépôt, et une application qui lit dans Vault des identifiants de base de données à durée de vie courte.

#	CHAPITRE	CONTENU	FORMAT	HEURES
3.1	Sécurité de l'IaC	Checkov, analyse IaC avec Trivy (anciennement tfsec), correction des erreurs de configuration dans Terraform et les manifestes Kubernetes	Cours + TP	3 h
3.2	Politiques en code	Kyverno en TP, comparé à OPA Gatekeeper et à ValidatingAdmissionPolicy intégré à Kubernetes ; écriture et test de politiques, modes audit et blocage, vérification des images avec Kyverno verifyImages	Cours + TP	3 h
3.3	HashiCorp Vault	Architecture, moteurs de secrets, secrets dynamiques, politiques et authentification	Cours + TP	3 h
3.4	Les secrets dans la chaîne	Intégration de Vault avec Jenkins et Kubernetes, détection de secrets avec gitleaks, épingleage des actions tierces par SHA de commit	Cours + TP	3 h

Kubernetes et exécution

Les participants durcissent le cluster qui exécute l'application du cours avec les Pod Security Standards, un RBAC au moindre privilège et des NetworkPolicies qui refusent tout par défaut, puis détectent les attaques en cours avec des règles Falco. TP validé : l'ouverture d'un shell dans un conteneur en fonctionnement détectée par Falco et une alerte levée.

#	CHAPITRE	CONTENU	FORMAT	HEURES
4.1	Durcissement de Kubernetes	Pod Security Standards, durcissement RBAC, NetworkPolicies	Cours + TP	3 h
4.2	Sécurité à l'exécution	Règles Falco, détection et alertes à l'exécution	Cours + TP	3 h

PHASE 05

9 H

Projet final

Par équipes de deux ou trois, les participants sécurisent la chaîne de livraison d'une application fournie, en réutilisant les TP de phase, puis présentent le résultat et ses risques résiduels comme ils le feraient devant un responsable de la sécurité.

#	CHAPITRE	CONTENU	FORMAT	HEURES
5.1	Cahier des charges et modèle de menaces	Analyser le besoin, modéliser les menaces, choisir les contrôles et leurs seuils	Projet	3 h
5.2	Construire la chaîne sécurisée	SAST, SCA, DAST, analyse et signature des images, analyse de l'IaC, secrets dans Vault	Projet	3 h
5.3	Exécution et rapport	Politiques du cluster et règles Falco ; rapport de sécurité avec vulnérabilités, corrections et risques acceptés	Projet	1,5 h
5.4	Démonstration et revue	Démonstration en direct d'une modification vulnérable arrêtée par la chaîne ; questions et retours	Projet	1,5 h

PROJET FINAL**Une chaîne de livraison sécurisée**

Une chaîne avec un contrôle de sécurité à chaque étape : code, dépendances, application en fonctionnement, images, code d'infrastructure, secrets et exécution sur Kubernetes, des images signées admises dans le cluster, et un rapport de sécurité qui précise ce qui est corrigé et le risque qui reste.

OUTILS GitHub Actions ou Jenkins, SonarQube, Semgrep, OWASP Dependency-Check, ZAP, Trivy, Syft, Cosign, Checkov, Kyverno ou OPA Gatekeeper, HashiCorp Vault, gitleaks, Falco, Kubernetes

LIVRABLE Dépôt avec la chaîne et les politiques, modèle de menaces, rapport de sécurité, démonstration en direct